<meta> should minimize standard library dependencies

Document #:	P3429R1
Date:	2024-11-15
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Jonathan Müller (think-cell) <foonathan@jonathanmueller.dev>

Contents

1 Abstract

[P2996R8 - Reflection for C++26] requires library support in the form of the `<meta>` header. As specified, this library API requires other standard library facilities such as `std::vector`, `std::string_view`, and `std::optional`. We propose minimizing these standard library facilities to ensure more wide-spread adoption. In our testing, the proposed changes have only minimal impact on user code.

2 Revision History

2.1 R0

Initial revision.

2.2 R1

- Changed the number of typos.
- Rebase on top of P2996R8; removing now redundant polls 1.3, 3, and 4.1.
- Expanded motivation.
- Discussed alternatives to `info_array`.

3 Background

[P2996R8] adds a new `<meta>` header for reflection library support. The functions introduced there are essentially wrappers around compiler built-ins and thus cannot be implemented by users: People that want to use reflection have to use `<meta>` or directly reach for the compiler built-ins. `<meta>` thus has the same status as `<type_traits>`, `<coroutine>`, `<initializer_list>`, and `<source_location>`. Yet, unlike those headers, which carefully avoided standard library dependencies, `<meta>` does not.

Right now, it is specified to include:

- `<initializer_list>`
- `<ranges>`
- `<string_view>`
- `<vector>`

In addition, the interface requires:

- `std::size_t`, `std::ptrdiff_t` in various places
- `<compare>` (for `member_offsets` defaulted comparison operator)
- `std::optional` (in `data_member_options_t`)

This means every implementation of `<meta>` is required to include the specified headers, and provide the definitions for the specified types. In addition, the compiler needs to be able to construct and manipulate objects of various standard library types at compile-time.

4 Motivation

The C++ standard can be split into three parts:

- The language, which has to be implemented by the compiler.
- The "normal" standard library, which is implemented by the standard library vendor, but can be re-implemented by anyone.
- The language support library, which requires co-operation with the compiler, so can only reasonable be implemented by the standard library vendor. This includes `<type_traits>`, `<coroutine>`, `<initializer_list>`, `<source_location>`, and now `<meta>`.

The existence of the third category means that in order to fully use all C++ language features, you not only need a C++ compiler, but also some standard library implementation to essentially provide a different spelling for language features. This is not too bad for small headers

like `<initializer_list>`, but `<meta>` as specified right now pulls in a lot of standard library machinery. And in a way, any standard library feature included by `<meta>` (and other language support headers) is itself elevated to the language support library: To make use of `<meta>`, you also need a (compile-time) implementation of `std::vector`, `std::string_view`, and `std::optional`. This couples more the standard library and the language even more than it already is and makes it harder to use C++ without the standard library.

We thus propose that `<meta>` should minimize standard library dependencies. This has the following advantages:

— **We serve all our users:** Developers in some domains (e.g. gamedev, embedded) make heavy use of C++ but avoid using the standard library. It is not for us in the committee to say whether they are justified in their reasons, but we have a duty to represent the interests of all our users and not just the subset of users that have representation in WG21. If we can better support everyone at minimal cost, we should do so or risk seeming even more out of touch.

— **Better compile-times due to reduced header inclusion:** A translation unit that does not require `<ranges>` or `<vector>` should not be forced to pay the (significant!) price for including them. This is especially important as reflection by design has to be used in a header file, and we anticipate wide-spread use of reflection in core utility libraries included in most translation units. If this comes with the entirety of `<ranges>`, this can cause significant increases in compile-time for projects that have not adopted it.

  For example, [lexy] is a C++ parser combinator library which avoids using standard library headers in the core library code as the metaprogramming alone makes compile-times slow enough. Right now, a clean rebuild takes 61.979 s ± 1.646 s. Including `<ranges>`, `<vector>` and `<string_view>` in a core header file, where reflection might be used to replace `__PRETTY_FUNCTION__` hacks, increases it to 86.786 s ± 0.468 s. Actually starting to use reflection features will slow it down further. This is makes reflection unusable.

  Modules may solve this problem in the long-term, but adopting modules requires changes to the build system, which makes it more difficult than "just" updating the compiler to start using reflection. It is not entirely unreasonable to think that some companies will start using reflection before they start using modules. In addition, some libraries will have to support both module and non-module builds for a long time, but can eagerly adopt reflection by simply querying for the feature test macro.

— **Better compile-times due to simpler API types:** All reflection APIs are `consteval`, so if a function e.g. returns a `std::vector`, the compiler has to construct this object at compile-time. This is expensive, as the memory layout and behavior of `std::vector` is controlled by standard library implementations the compiler frontend does not necessarily have full control over. If the result type were a custom type in control of the compiler instead, compiler implementers can pick an efficient representation that is easier to work with at compile-time.

  As a concrete example, `std::meta::members_of()` returns a `std::vector<std::meta::info>`. In the [bloomberg-clang] implementation, the underlying compiler API essentially exposes a `begin()` + `next()` function that is wrapped into a range type and passed to `std::vector`'s constructor. The compile-time interpreter then needs to allocate compile-time memory and simulate iterator machinery to construct a `std::vector` object. We propose that the API instead returns a `std::meta::info_array` whose layout is completely in control by the compiler. Then the compiler can allocate an array of `std::meta::info` objects using its own implementation, and not C++ code in an interpreter, and simply expose a pointer to that array.

— **You do not pay for what you do not use:** Most use cases of e.g. `std::meta::members_of()` just call it and iterate over it. Yet they have to pay for a full copy of the member list living in a `std::vector` that is immediately destroyed afterwards. This copy is necessary in the general case as querying properties of an entity at different points in a translation unit can result in different results (e.g. due to incomplete types that become complete). But if you immediately iterate over it, the copy is an unnecessary cost.

  With a custom result type, a smart compiler implementation can employ copy-on-write techniques to avoid unnecessary copies.

— **Usable in more contexts:** At think-cell, we overwrite the `_ASSERT` macros from the Windows APIs to use our custom assertion reporting mechanism. This is done by re-defining the macros before any Windows headers are included that would define `_ASSERT` themselves. Our definition of `_ASSERT` thus must not itself include any Windows headers, which means we have to be very careful about the standard library headers it includes. For example, we cannot include headers like `<ranges>` or `<string_view>`. If `<meta>` includes them, we cannot use reflection in our assertion definition, which is something we would like to do.

  Note that there is not any way to hide a dependency of `<meta>`: As it requires templates, any use of reflection has to be in a header file. And as we want to define a macro, we cannot use modules either. If `<meta>` pulls in a header that already defines `_ASSERT`, we cannot use reflection there.

  We are probably not the only ones that have a use case like this. Minimizing the standard library dependencies is the only option.

— **Precedence:** `std::source_location::file_name()` does not return a `std::string_view`, but a `const char*` instead. Proposed `std::contracts::contract_violation::comment()` does not return a `std::string_view`, but a `const char*` instead. So why should `std::meta::identifier_of()` return a `std::string_view`?

— **Minimal downsides:** The proposed changes have minimal negative impact on user code, so the cost of applying them is not high.

We therefore propose that `<meta>` should minimize standard library dependencies. In this paper, we exhaustively enumerate every dependency it has and suggest an alternative. Note that we are not proposing to force the implementation to avoid standard library dependencies in their implementations; we are just making it possible for them to do so.

In general, we hope that this would lead to more awareness when designing language support library features and treat them differently from the rest of the standard library. Ideally, a standard library facility that requires compiler support should not use a standard library facility that does not, yet it unnecessary bloats the language support subset of the standard library. We hope that this paper provides precedence for a future LEWG policy to that effect.

§ 5 Implementation experience

For the purposes of testing the impact on user code, some of the proposed changes have been implemented by modifying the `<meta>` header of [bloomberg-clang]. These changes are:

— Poll 2: Replacing `std::vector` by `std::meta::info_array`. The implementation still uses `std::vector` internally, but the interface impact becomes visible.

— Poll 4.2: Replacing `std::[u8]string_view` in return positions by `const char[8_t]*`.

Poll 1.1 has obvious user impact and does not need to be investigated; polls 1.2, 1.3, and 4.1 are non-breaking changes that widen the interface contract; poll 3 is potentially a breaking change but requires compiler support to implement; poll 5 is an obvious breaking change that provides equivalent convenience and does not need to be investigated.

The modified `<meta>` header is available here: [prototype-impl]

We investigated the following examples from [P2996R8] by replacing the `#include <experimental/meta>` with an `#include` of the above implementation:

— enum to string (no change needed)

— parsing command-line options (no change needed)

— a simpler tuple type (no change needed)

— a simpler variant type (no change needed)

— struct to struct of arrays (replace hard-coded type by `auto`)

— parsing command-line options II (no change needed)

— a universal formatter (no change needed)

— converting a struct to tuple (no change needed)

— implementing tuple_cat (note: required adjustment unrelated to this paper)

— named tuple (no change needed)

Similarly, we investigated the following examples of [daveed-keynote]:

— tuple implementation (no change needed)

— command-line argument parsing (additional `std::ranges::to<std::vector>` needed)

We also manually investigated the reflection implementation of [daw_json_link], which requires an additional `std::ranges::to<std::vector>` in the implementation of a reflection function that has since been added to [P2996R8].

All wording in this paper is relative to [P2996R8].

§ 6 Baseline: Depending on `<initializer_list>`, `<compare>` and `std::size_t` is perfectly fine

Those facilities are other compiler interface headers that are perfectly fine to use. We do not propose removing those dependencies.

§ 7 Poll 0: Remove the list of includes from the wording

The wording requires an include of `<ranges>` but the interface only requires a couple of concepts. Yet, because it is specified in the wording, every implementation, even ones that partition their headers to avoid unnecessary dependencies like libc++, have to include `<ranges>`, and not just the smaller internal header that defines the necessary concepts.

Removing the requirement gives implementations more freedoms at the cost of users having to include the headers themselves when they need those features. But it is likely that users which heavily use e.g. `<ranges>` will have it included already anyway. However, users that don't use `<ranges>` will not have to pay the extra cost of the include. As noted in the motivation, these includes alone caused a 40% increase in compile-time for [lexy].

Requiring the include of `<initializer_list>` is not a big problem and is common for other headers.

7.1 User impact

Users that included `<meta>` now also need to ensure they have `<ranges>`, `<vector>`, or `<string_view>` included if they want to use declarations from those headers that aren't already used in `<meta>`'s interface.

7.2 Implementation impact

None. An implementation can still include whatever headers they want.

7.3 Wording

Modify the new subsection in 21 [meta] after 21.3 [type.traits]:

Header `<meta>` synopsis

```
#include <initializer_list>
-#include <ranges>
-#include <string_view>
-#include <vector>

namespace std::meta {
…
```

8 Range concepts

8.1 Poll 1.1: Make the `reflection_range` concept exposition-only

The wording defines a concept `reflection_range` that is used to constrain member functions. It is modeled by input ranges whose value and reference types are `meta::info` (references).

It is unlikely that users want to refine this concept further in a way that requires subsumption, so it is not necessary to expose it as a concept. So at best exposing the concept is convenient for users writing generic code which also requires input ranges whose value and reference types are `meta::info` (references). However, this problem is better solved by adding a generic `ranges::input_range_of<T>` concept, as it is a problem that is not specific to reflection.

Exposing the ad-hoc concept right now as-is would also freeze it in-place, so even if we had a `ranges::input_range_of<T>` concept, `reflection_range` would not subsume it. Leaving it exposition-only gives us more leeway.

8.1.1 User impact

Users that want to constrain a function on a range of `std::meta::info` objects need to write a similar concept themselves.

8.1.2 Implementation impact

None. An implementation presumably will still add the concept, just under a different name.

8.1.3 Wording

Modify the new subsection in 21 [meta] after 21.3 [type.traits]:

Header `<meta>` synopsis

```
…
  // [meta.reflection.substitute], reflection substitution
  template <class R>
-    concept reflection_range = see below;
+    concept reflection-range = see below; // exposition-only

-  template <reflection_range R = initializer_list<info>>
+  template <reflection-range R = initializer_list<info>>
    consteval bool can_substitute(info templ, R&& arguments);
-  template <reflection_range R = initializer_list<info>>
+  template <reflection-range R = initializer_list<info>>
    consteval info substitute(info templ, R&& arguments);
…
```

And likewise replace all others of `reflection_range` with *`reflection-range`*.

8.2 Poll 1.2: Change the *`reflection_range`* concept to match language semantics

As specified, *reflection_range* is a refinement of `ranges::input_range`, which imposes additional requirements on the iterator type, like the existence of a `value_type` and `difference_type` or `std::iterator_traits` specialization. Those requirements are not necessary for the range-based for-loop.

People that don't care about the standard library range concepts, still want to use reflection. They thus might have range types that don't model any of the standard library range concepts, but are still supported by the range-based for-loop. For an interface that is supposed to be low-level and close to the compiler, it can make sense to instead follow the language semantics, and not the standard library semantics.

§ 8.2.1 User impact

Positive, functions accept strictly more types than before.

Before

```
class MyRange
{
public:
    class iterator
    {
    public:
        using value_type      = std::meta::info;
        using difference_type = std::ptrdiff_t;

        iterator();

        std::meta::info operator*() const;
        iterator& operator++();
        void operator++(int);

        bool operator==(iterator, iterator) const;
    };

    iterator begin();
    iterator end();
};

auto result = substitute(info, MyRange{});
```

After

```
class MyRange
{
public:
    class iterator
    {
    public:

        std::meta::info operator*() const;
        iterator& operator++();

        bool operator==(iterator, iterator) const;
    };

    iterator begin();
    iterator end();
};

auto result = substitute(info, MyRange{});
```

§ 8.2.2 Implementation impact

Implementations need to write a custom concept instead of using `ranges::input_range`.

§ 8.2.3 Wording

Modify [meta.reflection.substitute] Reflection substitution

```
template <class R>
concept reflection_range =
-  ranges::input_range<R> &&
-  same_as<ranges::range_value_t<R>, info> &&
-  same_as<remove_cvref_t<ranges::range_reference_t<R>>, info>;
+  see-below; // exposition-only
```

A type `R` models the exposition-only concept *reflection-range* if a range-based for loop statement [stmt.ranged] `for (std::same_as<info> auto _ : r) {}` is well-formed for an expression of type `R`.

[*Note*: This requires checking whether the `begin-expr` and `end-expr` as defined in [stmt.ranged] are well-formed and that the resulting types support `!=`, `++`, and `*` as needed in the loop transformation. — *end note*]

§ 8.3 Poll 1.3

Made redundant by P2996R8.

§ 9 Poll 2: Replace `std::vector` by new `std::meta::info_array`

Functions that return a range of `meta::info` objects do it by returning a `std::vector<std::meta::info>` (for implementation reasons, it has to be an owning container and cannot be something like a `std::span`). For the reasons discussed above, this is not ideal.

Instead, all functions that return a `std::vector<std::meta::info>` should instead return a new type `std::meta::info_array`. This is a type that can only be constructed by the implementation and has whatever internal layout is most appropriate for the implementation.

Unlike `std::vector`, the proposed `std::meta::info_array` is not mutable and cannot grow in size. This simplifies the implementation further.

For the vast majority of calls, this change is not noticeable: All they do is iterate over the result, compose it with other views, or apply range algorithms. For those users that do rely on having a mutable, growable container, they need to call `std::ranges::to<std::vector>`.

§ 9.1 User impact

Users that want to append elements to a range of input objects or remove them from a range of input objects need to either use `views::concat`/`views::filter` or `std::ranges::to`.

In our testing, this only affected the command-line argument parsing example of Daveed's keynote [daveed-keynote]:

Before

```
static consteval auto
        clap_annotations_of(info
        dm) {
    auto notes =
        annotations_of(dm);
    std::erase_if(notes, [](info
        ann) {
        return
        parent_of(type_of(ann))
        != ^^clap;
    });
    return notes;
}
```

After (option 1)

```
static consteval auto
        clap_annotations_of(info dm)
        {
    auto notes = annotations_of(dm)
        |
        std::ranges::to<std::vector>
        ();
    std::erase_if(notes, [](info
        ann) {
        return
        parent_of(type_of(ann)) !=
        ^^clap;
    });
    return notes;
}
```

After (option 2)

```
static consteval auto
        clap_annotations_of(info
        dm) {
    return annotations_of(dm)
        | std::views::filter([]
        (info  ann) {
            return
        parent_of(type_of(ann))
        == ^^clap;
        });
}
```

A similar change would be needed for `daw_json_link`. However, their use case is served by `std::meta::get_public_nonstatic_data_members()`.

For completeness, we also needed to update the implementation of e.g. `std::meta::subobjects_of` which called `.append_range()` internally:

Before

```
consteval auto subobjects_of(info r) -> vector<info> {
    if (is_namespace(r))
        throw "Namespaces cannot have subobjects";

    auto subobjects = bases_of(r);
    subobjects.append_range(nonstatic_data_members_of(r));
    return subobjects;
}
```

After

```
consteval auto subobjects_of(info r) -> info_array {
    if (is_namespace(r))
        throw "Namespaces cannot have subobjects";

    auto subobjects = bases_of(r) | ranges::to<vector>();
    subobjects.append_range(nonstatic_data_members_of(r));
    return subobjects;
}
```

An implementation that does not use `std::vector` internally would not need to change anything.

Users that do not wish to modify the container and only iterate over it are not affected.

§ 9.2 Implementation impact

An implementation that does not care about decoupling dependencies just need to provide a wrapper class over `std::vector`. In our prototype implementation, it was done in 30 lines of code. A production-ready implementation can then also optimize `std::ranges::to` to avoid additional memory allocations and copies when the user wants a `std::vector`.

§ 9.3 Alternative designs

The proposed wording adds all members of `std::array`, except for `fill` (which doesn't make sense) and `reverse_iterator` (would introduce more standard library dependencies). That way it also provides all functions provided by `std::ranges::view_interface` for a contiguous range (except for `operator bool`, which is weird for a container). Alternatively, the minimum interface could model `std::initializer_list` and only provide `begin`/`end` and `size`.

Instead of specifying a new type `std::meta::info_array`, the return type of `std::meta::members_of` and co could be an implementation-defined type guaranteed to model some concepts (for example `std::ranges::contiguous_range`). Then an implementation can simply return `std::vector` directly if it does not care about dependencies. The downside of this approach is that user might rely on the existence of specific member functions that are not guaranteed and accidentally write non-portable code.

The author does not have a strong opinion on either alternative but prefers them all over returning `std::vector` directly.

§ 9.4 Wording

Modify the new subsection in 21 [meta] after 21.3 [type.traits]:

Header `<meta>` synopsis

```
…
namespace std::meta {
    using info = decltype(^::);

+    // [meta.reflection.info_array], info array
+    class info_array;

    …

-    consteval vector<info> template_arguments_of(info r);
+    consteval info_array template_arguments_of(info r);

    // [meta.reflection.member.queries], reflection member queries
-    consteval vector<info> members_of(info r);
+    consteval info_array members_of(info r);
-    consteval vector<info> bases_of(info type);
+    consteval info_array bases_of(info type);
-    consteval vector<info> static_data_members_of(info type);
+    consteval info_array static_data_members_of(info type);
-    consteval vector<info> nonstatic_data_members_of(info type);
+    consteval info_array nonstatic_data_members_of(info type);
-    consteval vector<info> enumerators_of(info type_enum);
+    consteval info_array enumerators_of(info type_enum);

-    consteval vector<info> get_public_members(info type);
+    consteval info_array get_public_members(info type);
-    consteval vector<info> get_public_static_data_members(info type);
+    consteval info_array get_public_static_data_members(info type);
-    consteval vector<info> get_public_nonstatic_data_members(info type);
+    consteval info_array get_public_nonstatic_data_members(info type);
-    consteval vector<info> get_public_bases(info type);
+    consteval info_array get_public_bases(info type);
}
```

Add a new section **[meta.reflection.info_array] Info array**:

```
class info_array {
public:
    using value_type             = info;
    using pointer                = const info*;
    using const_pointer          = pointer;
    using reference              = const info&;
    using const_reference        = reference;
    using size_type              = size_t;
    using difference_type        = ptrdiff_t;
    using iterator = pointer;

    info_array() = delete;
    ~info_array();
    constexpr info_array(const info_array&);
    constexpr info_array(info_array&&) noexcept;
    constexpr info_array& operator=(const info_array&);
    constexpr info_array& operator=(info_array&&) noexcept;

    constexpr void swap(info_array&) noexcept;

    constexpr iterator begin() const noexcept;
    constexpr iterator end() const noexcept;

    constexpr iterator cbegin() const noexcept { return begin(); }
    constexpr iterator cend() const noexcept { return end(); }

    constexpr bool empty() const noexcept { return begin() == end(); }
    constexpr size_type size() const noexcept { return end() - begin(); }
    constexpr reference operator[](size_type i) const noexcept { return begin()[i]; }
    constexpr reference front() const noexcept { return begin()[0]; }
    constexpr reference back() const noexcept { return end()[-1]; }
    constexpr pointer data() const noexcept { return begin(); }
};
```

[1] The type `info_array` is a non-mutable, non-resizable container of `info` objects.

etc. etc.

Update the other sections accordingly to use `info_array` instead of `vector<info>`.

10 Poll 3

Made redundant by P2996R8.

11 Replace `std::[u8]string_view`

11.1 Poll 4.1

Made redundant by P2996R8.

11.2 Poll 4.2: Replace `std::[u8]string_view` as return type with `const char[8_t]*`

`[u8]identifier_of`, `[u8]symbol_of` and `[u8]display_string_of` return a `std::[u8]string_view`. In addition, to the dependency problems, it is not guaranteed to be a null-terminated string, and even if it were, getting a null-terminated string out of a `std::string_view` requires an awkward `.data()` call and a comment explaining why it is null-terminated. Both problems are solved by returning a `const char[8_t]*` instead, just like `std::source_location::file()` does, which is a very similar function.

The downside is that users who want one of the gazillion member functions have to manually create a `std::string_view` first. However, most calls probably forward the resulting identifier unchanged and are unaffected; in our testing we have not found any. All the examples treat the identifier as an opaque blob that is being forwarded to some mechanism that handles `const char*` just fine.

11.2.1 User impact

Users that require a `std::[u8]string_view` as opposed to a `const char[8_t]*` need to construct it manually. In our testing we have not found any.

Before	After
`auto name = identifier_of(info);` `… name.find(…) …`	`std::string_view name = identifier_of(info);` `… name.find(…) …`

Users that require a null-terminated string get one directly.

11.2.2 Implementation impact

None, the compiler API on clang for example already returns a `const char[8_t]*`.

11.2.3 Wording

Modify the new subsection in 21 [meta] after 21.3 [type.traits]:

Header `<meta>` synopsis

```
…
    // [meta.reflection.operators], operator representations
    …
-   consteval string_view symbol_of(operators op);
+   consteval const char* symbol_of(operators op);
-   consteval u8string_view u8symbol_of(operators op);
+   consteval const char8_t* u8symbol_of(operators op);

    // [meta.reflection.names], reflection names and locations
    consteval bool has_identifier(info r);

-    consteval string_view identifier_of(info r);
+    consteval const char* identifier_of(info r);
-    consteval u8string_view u8identifier_of(info r);
+    consteval const char8_t* u8identifier_of(info r);

-    consteval string_view display_string_of(info r);
+    consteval const char* display_string_of(info r);
-    consteval u8string_view u8display_string_of(info r);
+    consteval const char8_t* u8display_string_of(info r);
```

```
    consteval source_location source_location_of(info r);
…
```

And update [meta.reflection.operators] and [meta.reflection.names] accordingly.

§ 12 Poll 5: Re-design `data_member_spec`

`data_member_spec` defines a new data member. It only takes one mandatory attribute, the type, and optional options in an object of type `data_member_options_t`. This aggregate type has multiple standard library dependencies:

1. The members `name`, `alignment` and `width` are `std::optional`.
2. The nested type `name_type` is specified to be constructible from anything a `std::string` or `std::u8string` can be constructed from. This requires at least knowledge of the constructors, although an implementation could do heroics to depend pulling in the header.

Ignoring the dependencies, the proposed design has multiple other problems that could be fixed:

— The `name` member, if present, stores a copy of a user-provided string, which is then further copied into the compiler internal storage that represents a data member specification when calling `data_member_spec`.

— It is an error to specify both `alignment` and `width` yet the API does nothing to prevent that.

A design that instead provides multiple creation functions combined with setters has none of those problems.

§ 12.1 Proposed Design

```
class data_member_spec_t {
public:
    consteval data_member_spec_t& name(/* depends on polls 4.1 and 1.3 */ name); // set the name
    consteval data_member_spec_t& no_unique_address(bool enable = true); // set no_unique_address

    consteval operator info() const; // build the data member specification
};

consteval data_member_spec_t data_member_spec(info type); // unnamed, unaligned member with no attributes
consteval data_member_spec_t data_member_spec_aligned(info type, int alignment; // unnamed, aligned member with no
        attributes
consteval  data_member_spec_t  data_member_spec_bitfield(info  type,  int  width);  //  unnamed  bitfield  with  no
        attributes
```

We provide named functions to create the three different cases of data members. They return a builder object that can be further modified with setters and implicitly converted to an `info` object.

An implementation of `data_member_spec_t` that guards against ABI breaks can just store a single `info` object that represents the data already given, and modifies the compiler internal representation when calling `.name()` and `.no_unique_address()`.

This requires also changes to `define_aggregate` to accept a range of types convertible to `info`.

§ 12.2 User impact

The API changes dramatically, but it does not affect the convenience in any way.

Before	After
`define_aggregate(^storage, {data_member_spec(^T, {.name = "foo", .no_unique_address = true})})`	`define_aggregate(^storage, {data_member_spec(^T).name("foo").no_unique_address()})`

§ 12.3 Implementation impact

An implementation that does not care about minimizing dependencies can implement `data_member_spec_t` in terms of the current `data_member_options_t`.

§ 12.4 Wording

TBD

§ 13 References

[bloomberg-clang] Bloomberg's clang implementation of P2996.
https://github.com/bloomberg/clang-p2996
[daveed-keynote] Daveed Vandevoorde's closing CppCon2024 keynote.
http://vandevoorde.com/CppCon2024.pdf
[daw_json_link] daw_json_link.
https://github.com/beached/daw_json_link/blob/85f5f3f3d15a27fa000733f758b157d2267a74c8/include/daw/json/daw_json_reflection.h
[lexy] lexy.
https://github.com/foonathan/lexy
[P2996R8] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. Reflection for C++26.
https://wg21.link/P2996R8
[prototype-impl] prototype implementation.
https://gist.github.com/foonathan/457bd0073cfde568e446eb4d42ec87fe